SDN-Based DoS Attacks and Mitigation Project

Student Name: Cuneyt Terzi

Email: cterzi@asu.edu

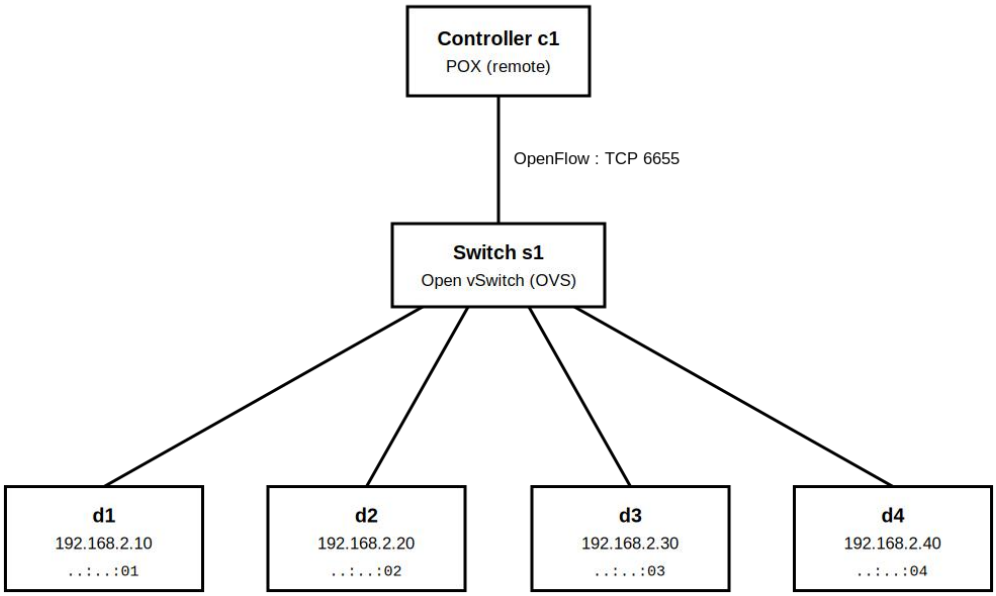
Submission Date: 6/21/2026

Class Name and Term: CSE548 Summer 2026

I. Project Overview

As the data plane and control plane are separate in an SDN, some devices in the network initially do not have the knowledge of all the policies and need to forward the incoming packets to the controller to get the routing policy of that specific source-destination pair. This creates a gap in security where the system can be bogged down if a big number of packets could be forced to forward to the controller, resulting in an unresponsive, resource deprived system. To mitigate such attacks, the firewall logic in the script file can be modified to detect such a pattern that would result in forwarding too many packets to the controller. In this project, I will demonstrate the attack being successful and how the logic can be enhanced to prevent these attacks by observing any unusual source-destination relationships and blocking any source MAC address that is associated to more than one IP address, which is a sign of spoofed packets. Once a drop rule is added for that address, the rest of the packets will be dropped without ever being forwarded to the contoller.

II. Network Setup



III. Software

- Ubuntu 22.04 LTS (ARM64) virtual machine.
- Mininet network emulator
- Docker
- Open vSwitch (OVS) — the OpenFlow-enabled data-plane switch (s1)
- POX
- ovs-ofctl / ovs-vsctl — Open vSwitch
- hping3

IV. Project Description

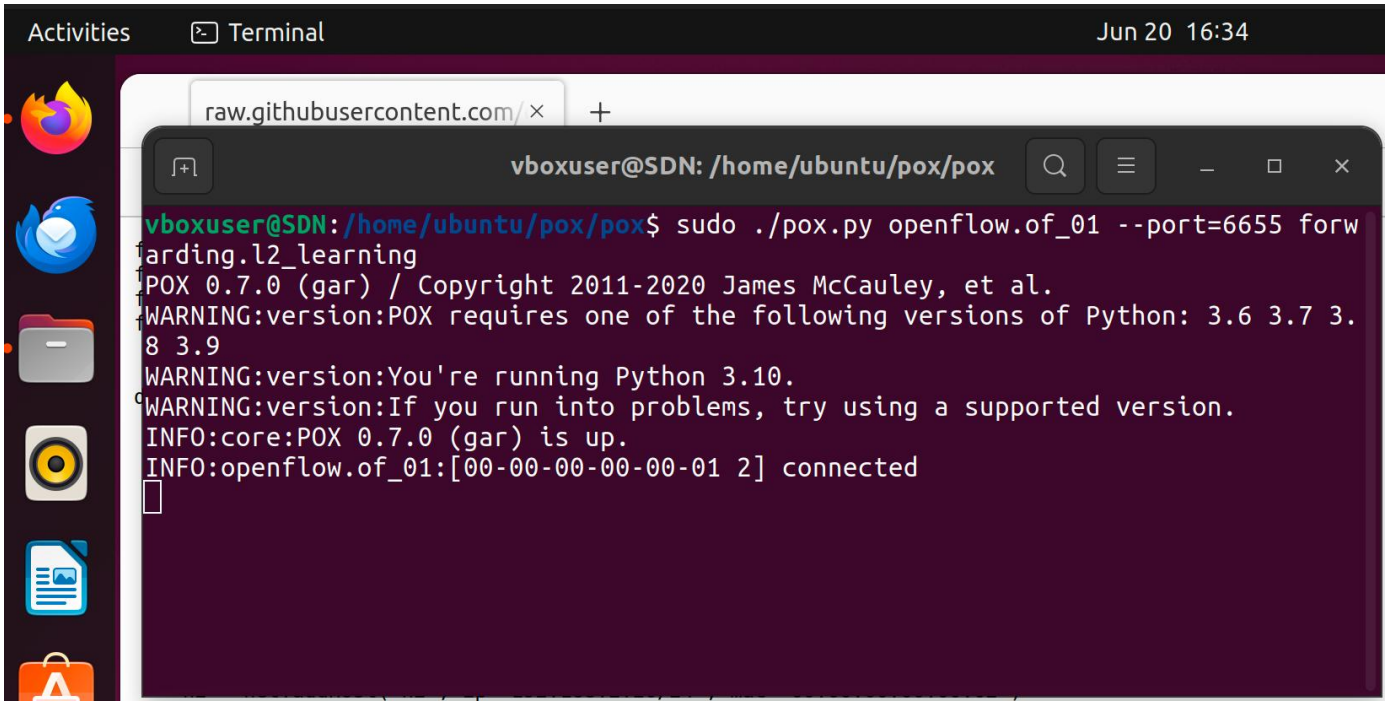
Step 1 : Copy project files

```
# topology script in the home directory
~/mininet_topo.py

# firewall component in the POX forwarding folder
~/pox/pox/forwarding/L3Firewall.py

# both config files in the POX root directory
~/pox/l2firewall.config
~/pox/l3firewall.config
```

Step 2 : Start POX Controller

We launch the POX controller from the command line first using the default l2_learning algorithm. This will not drop any packets by default and thus is vulnerable to DDOS attacks.

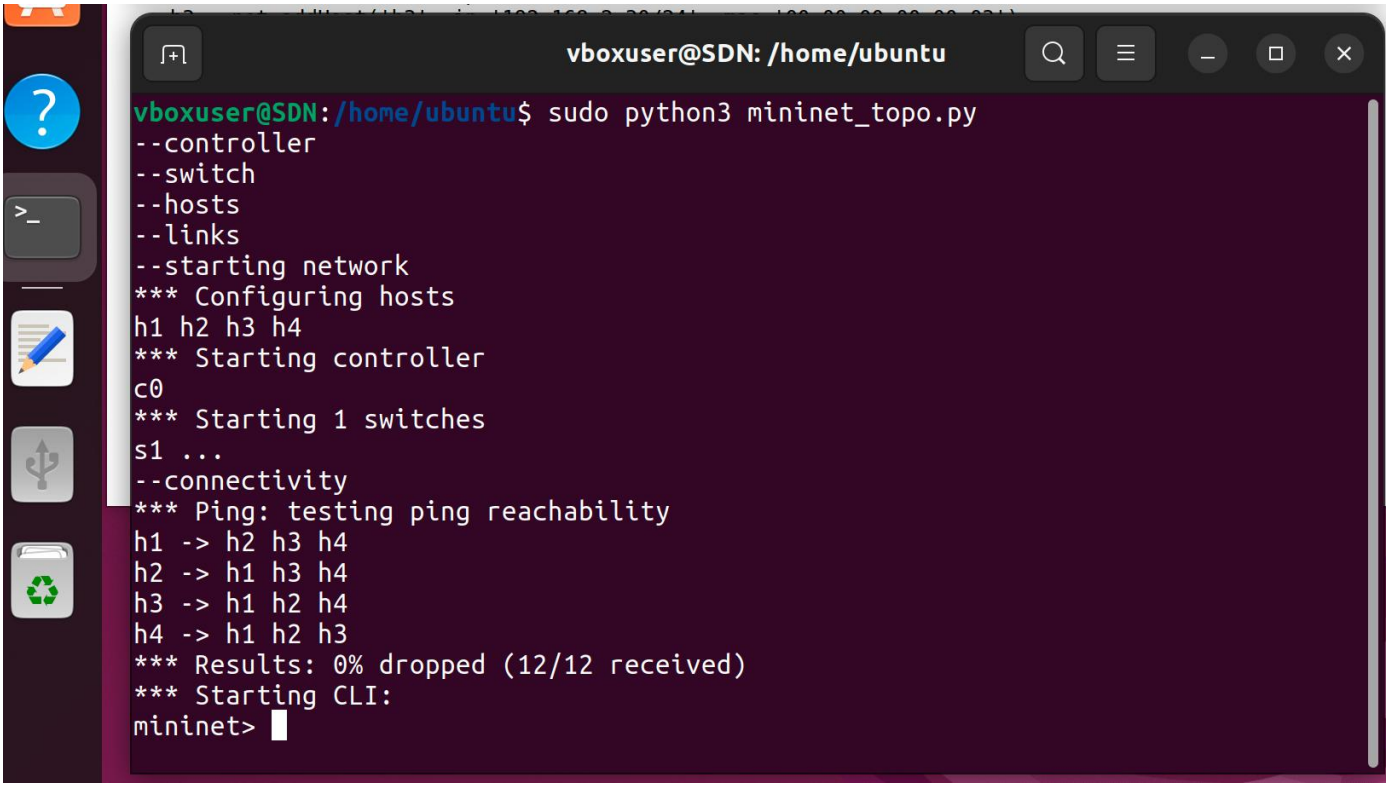
```
sudo ./pox.py openflow.of_01 --port=6655 pox.forwarding.l2_learning
```



Step 3 : Start Mininet network

We run the mininet_topo.py script that will create the objects that make up the mininet hosts and the switch.The execute permission on the py file was set after copying the file so we can directly execute it.

```
sudo python3 ./mininet_topo.py
```

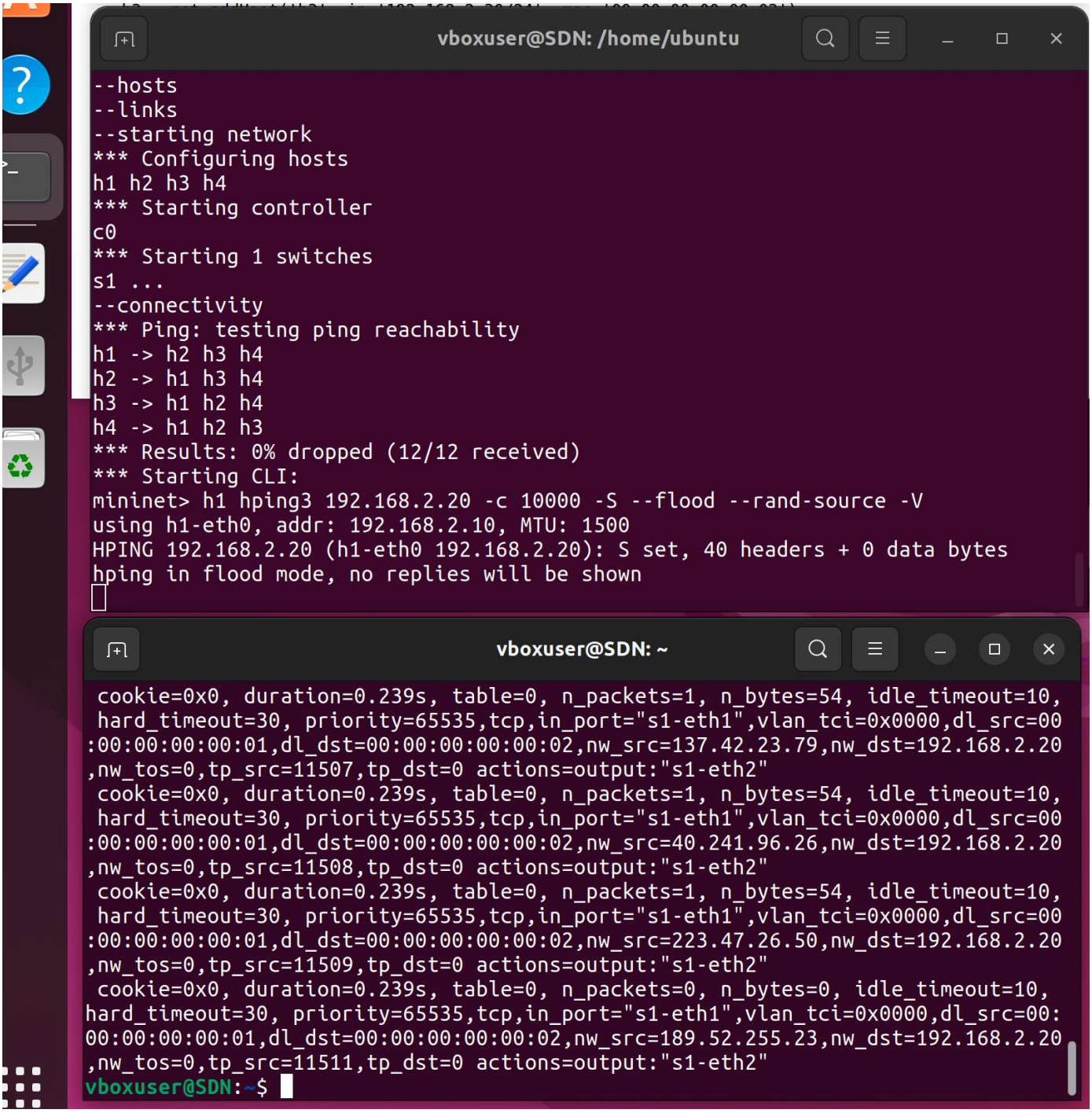


Step 4: Perform the Flood

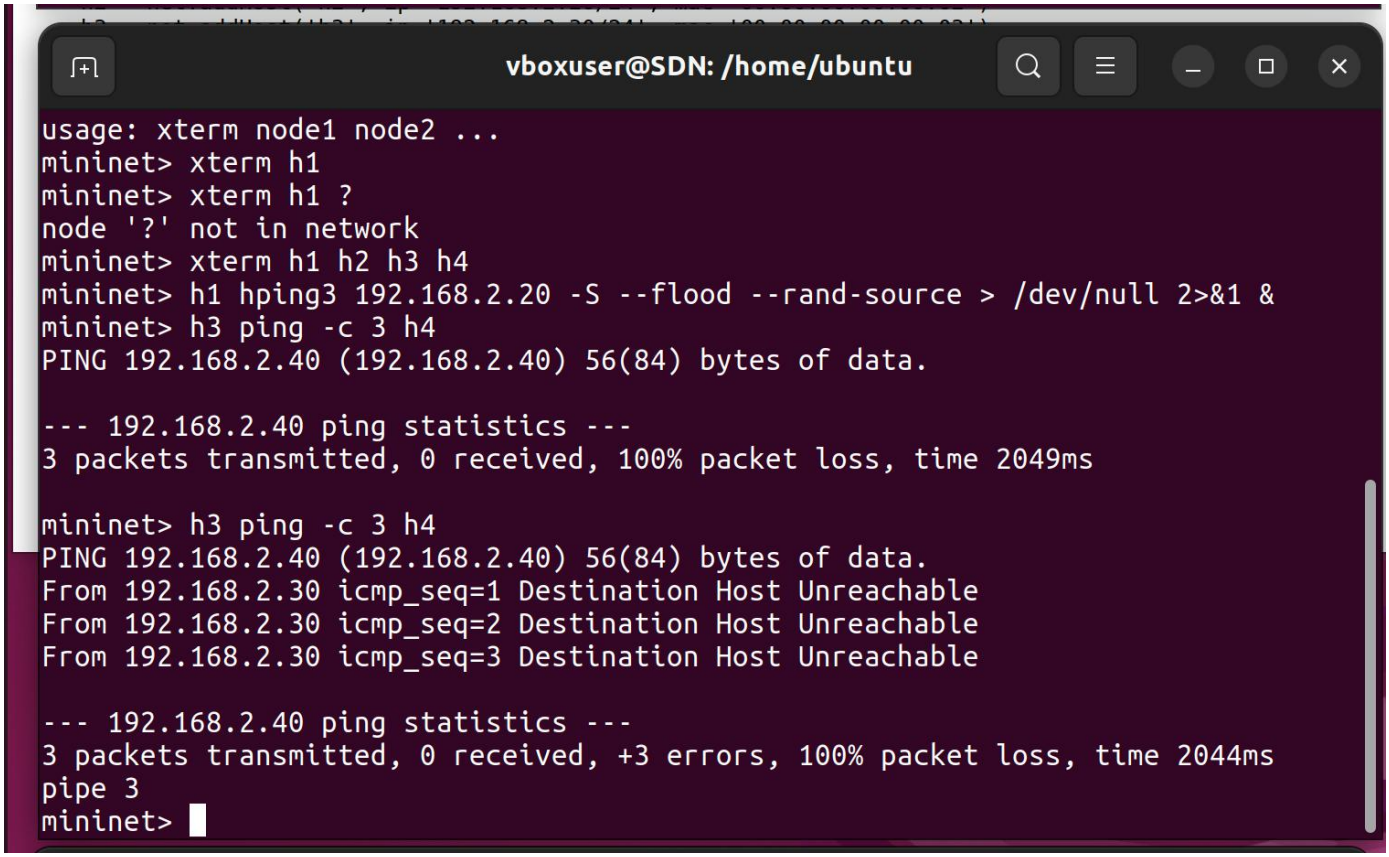
```
h1 hping3 192.168.2.20 -c 10000 -S --flood --rand-source -V
```

hping3 has two switches which make the ddos attack possible and they are -flood and -rand-source. -rand-source especially randomizes the source Ip of the packets it sends so they look like they come from different sources. this ensures each of these packets get sent to the conroller for any matching policies.

A ping from the other two hosts end up throwing the error that “Destination host unreachable’ which proves the point that the controller has been overflown with too many packets to process.



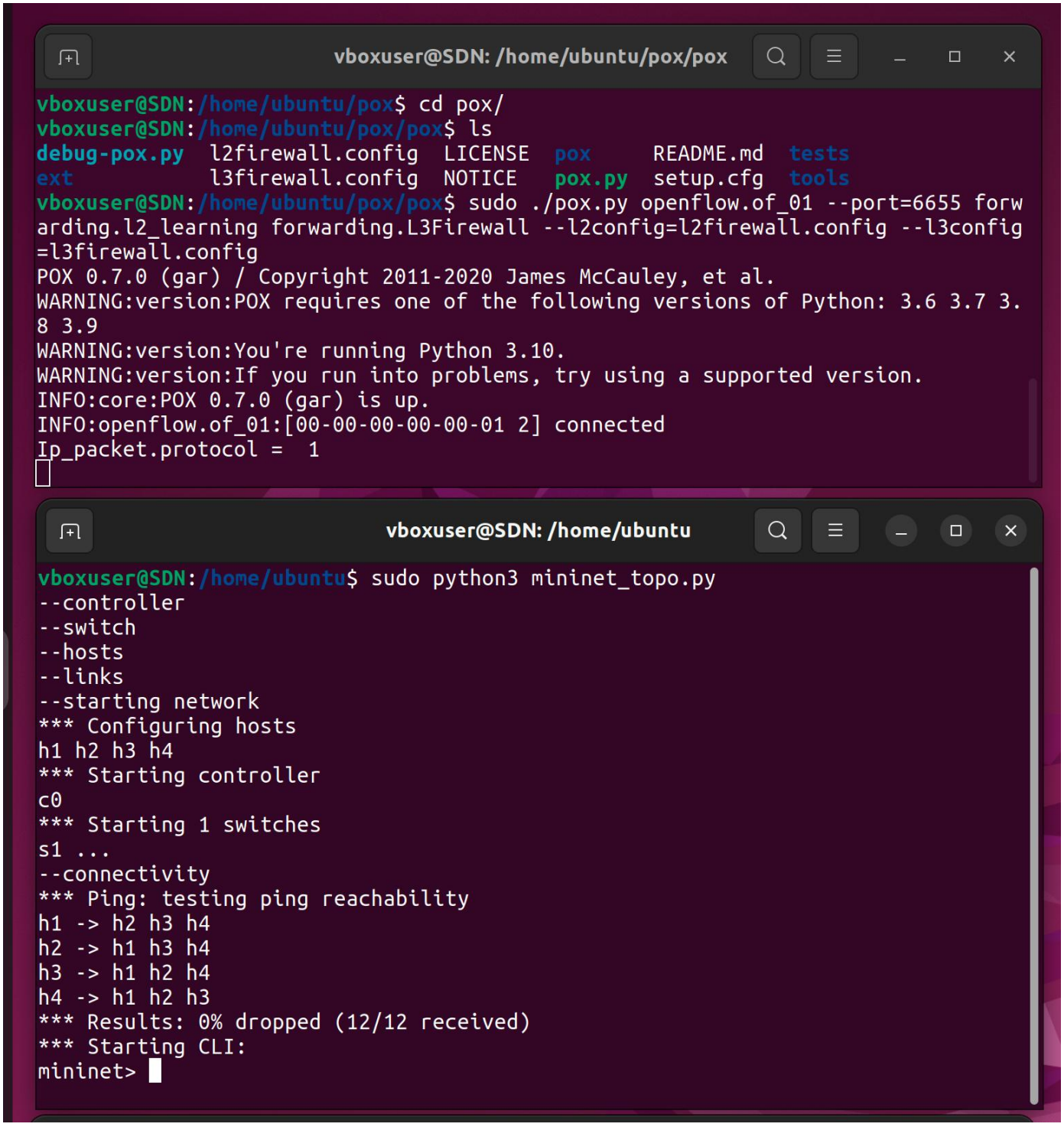
And trying to ping h4 from h3 will end up with “Destination host unreachable” error, as below.



Step 5 : Mitigate DoS Attack by implementing port security in accordance with pseucode posted in Task 3.0 in L3Firewall.py
Initiate a port table (PT);
For any newly received flow, F originated from the source MAC address F.SrcMAC;
if F.SrcIP is new;
update PT with the mapping F.SrcMAC <--> F.SrcIP;
else
block F.SrcMAC % Block a MAC address that had spoofed multiple IP addresses
end

For testing the mitigation code, we have updated the L3Firewall.py code with the above logic. It simply keeps track of a MAC address-sourceIp pair and makes sure each MAC address is only matched with one IP. The relevant portions of the code can be found below.

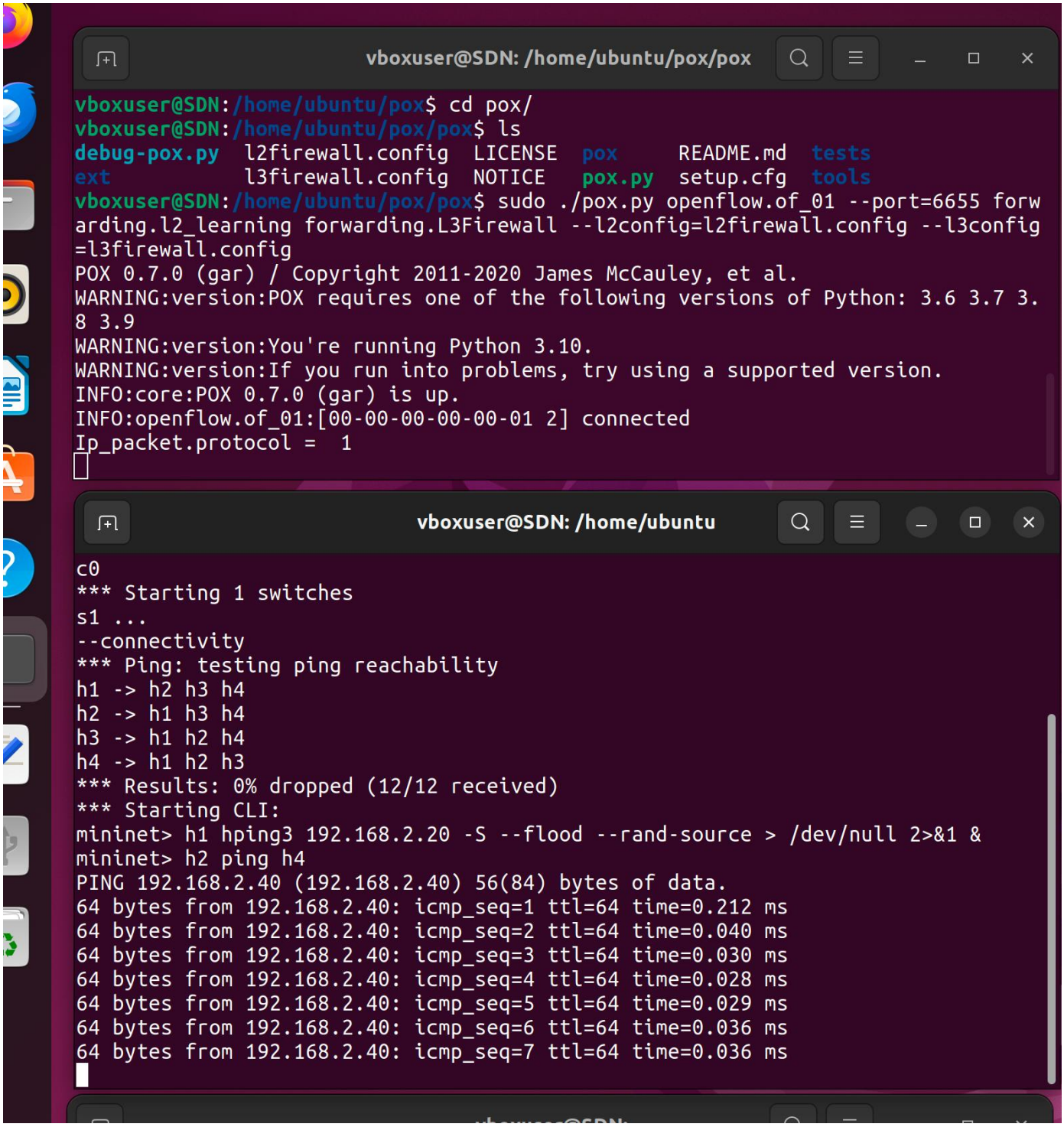
You can find below the mitigation code in action. After starting pox.py with the L2 and L3 config parameters and L3Firewall.py code, I started the mininet and it's ready for testing the flood.

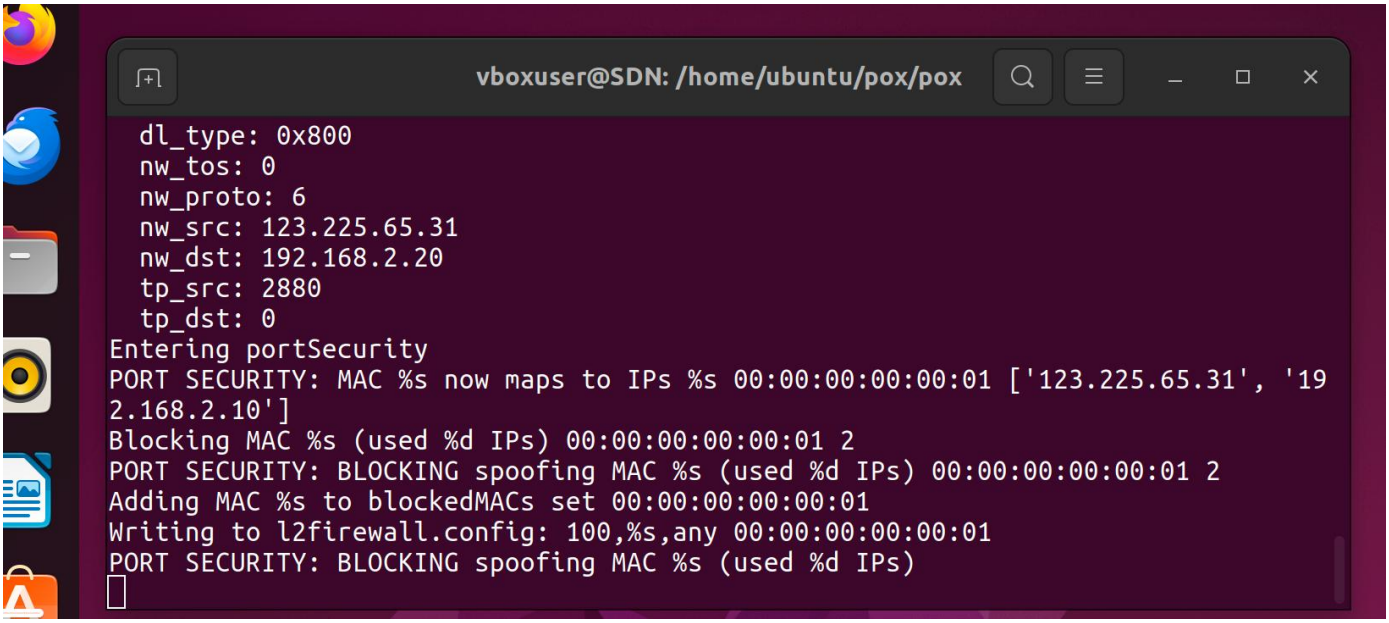


Start the flood in background

```
h1 hping3 192.168.2.20 -S --flood --rand-source > /dev/null 2>&1 &
```

This time the ping from h2 to h4 will work because of the blocking policy in place


vboxuser@SDN: /home/ubuntu/pox/pox
vboxuser@SDN:/home/ubuntu/pox$ cd pox/
vboxuser@SDN:/home/ubuntu/pox/pox$ ls
debug-pox.py l2firewall.config LICENSE pox   README.md tests
ext   l3firewall.config NOTICE   pox.py setup.cfg tools
vboxuser@SDN:/home/ubuntu/pox/pox$ sudo ./pox.py openflow.of_01 --port=6655 forwarding.l2_learning forwarding.L3Firewall --l2config=l2firewall.config --l3config=l3firewall.config
POX 0.7.0 (gar) / Copyright 2011-2020 James McCauley, et al.
WARNING:version:POX requires one of the following versions of Python: 3.6 3.7 3.8 3.9
WARNING:version:You're running Python 3.10.
WARNING:version:If you run into problems, try using a supported version.
INFO:core:POX 0.7.0 (gar) is up.
INFO:openflow.of_01:[00-00-00-00-00-01 2] connected
Ip_packet.protocol = 1
vboxuser@SDN: /home/ubuntu
c0
*** Starting 1 switches
s1 ...
--connectivity
*** Ping: testing ping reachability
h1 -> h2 h3 h4
h2 -> h1 h3 h4
h3 -> h1 h2 h4
h4 -> h1 h2 h3
*** Results: 0% dropped (12/12 received)
*** Starting CLI:
mininet> h1 hping3 192.168.2.20 -S --flood --rand-source > /dev/null 2>&1 &
mininet> h2 ping h4
PING 192.168.2.40 (192.168.2.40) 56(84) bytes of data.
64 bytes from 192.168.2.40: icmp_seq=1 ttl=64 time=0.212 ms
64 bytes from 192.168.2.40: icmp_seq=2 ttl=64 time=0.040 ms
64 bytes from 192.168.2.40: icmp_seq=3 ttl=64 time=0.030 ms
64 bytes from 192.168.2.40: icmp_seq=4 ttl=64 time=0.028 ms
64 bytes from 192.168.2.40: icmp_seq=5 ttl=64 time=0.029 ms
64 bytes from 192.168.2.40: icmp_seq=6 ttl=64 time=0.036 ms
64 bytes from 192.168.2.40: icmp_seq=7 ttl=64 time=0.036 ms



V. Conclusion

The most important lesson I learned from this assignment is that the seperation and the physical distance between the control mechanism and the execution device opens up an attack surface and it's relatively easy to create a successful DDOS attack using basic methods. This brings us to the concept of planing, code based configurations and review pipeline being very important.

Being able to use a modern programming language to be able to write logic gives the programmer a lot of power in terms of flexiblility defining the security policies.

The DDOS attack that is the goal of this project was mitigated by keeping track of the mac address - ip address pairs and immediately updates the drop rule in the switch to drop further packets from that mac address as soon as it detects the second source IP from the same mac address.

VI. Appendix B: Attached files

L3Firewall.py

```
from pox.core import core
import pox.openflow.libopenflow_01 as of
from pox.lib.revent import *
from pox.lib.util import dpidToStr
from pox.lib.addresses import EthAddr
from collections import namedtuple
import os

import csv
import argparse
from pox.lib.packet.ethernet import ethernet, ETHER_BROADCAST
from pox.lib.addresses import IPAddr
import pox.lib.packet as pkt
from pox.lib.packet.arp import arp
from pox.lib.packet.ipv4 import ipv4
from pox.lib.packet.icmp import icmp

log = core.getLogger()
priority = 50000

l2config = "l2firewall.config"
l3config = "l3firewall.config"


class Firewall (EventMixin):

    def __init__ (self,l2config,l3config):
```

```
    self.listenTo(core.openflow)
    self.disabled_MAC_pair = [] # Store a tuple of MAC pair which will be installed into the flow table
of each switch.
    self.fwconfig = list()

    # ------------------------------------------------------------------
    # PORT SECURITY lists and option
    self.portTable    = {}
    self.blockedMACs  = set()
    self.maxIPsPerMAC = 1

    if l2config == "":
      l2config = "l2firewall.config"

    if l3config == "":
      l3config = "l3firewall.config"

    with open(l2config, 'r') as rules:
      csvreader = csv.DictReader(rules) # Map into a dictionary
      for line in csvreader:
        # read MAC address
        if line['mac_0'] != 'any':
          mac_0 = EthAddr(line['mac_0'])
        else:
          mac_0 = None

        if line['mac_1'] != 'any':
          mac_1 = EthAddr(line['mac_1'])
        else:
          mac_1 = None
        # add pair to pair list
        self.disabled_MAC_pair.append((mac_0, mac_1))

    with open(l3config) as csvfile:
      print("Reading l3config file in __init__ ")
      self.rules = csv.DictReader(csvfile)
      for row in self.rules:
        print("Saving individual rule parameters in rule dict !")
        s ip = row['src ip']
        d_ip = row['dst_ip']
        s_port = row['src_port']
        d_port = row['dst_port']
        print ("src_ip, dst_ip, src_port, dst_port", s_ip, d_ip, s_port, d_port)

    print("Enabling Firewall Module")

    def replyToARP(self, packet, match, event):
    r = arp()
    r.opcode = arp.REPLY
    r.hwdst = match.dl_src
    r.protosrc = match.nw_dst
    r.protodst = match.nw_src
    r.hwsrc = match.dl_dst
    e = ethernet(type=packet.ARP_TYPE, src=r.hwsrc, dst=r.hwdst)
    e.set_payload(r)
    msg = of.ofp_packet_out()
    msg.data = e.pack()
    msg.actions.append(of.ofp_action_output(port=of.OFPP_IN_PORT))
    msg.in_port = event.port
    event.connection.send(msg)

    def allowOther(self, event):
    msg = of.ofp_flow_mod()
    match = of.ofp_match()
    action = of.ofp_action_output(port=of.OFPP_NORMAL)
    msg.actions.append(action)
    event.connection.send(msg)

    def installFlow(self, event, offset, srcmac, dstmac, srcip, dstip, sport, dport, nwproto):
    print("Entering installFlow")

    msg = of.ofp_flow_mod()
    match = of.ofp_match()
    if(srcip != None):
      match.nw_src = IPAddr(srcip)
    if(dstip != None):
```

```
    match.nw_dst = IPAddr(dstip)
  match.nw_proto = int(nwproto)
  match.dl_src = srcmac
  match.dl_dst = dstmac
  match.tp_src = sport
  match.tp_dst = dport
  match.dl_type = pkt.ethernet.IP_TYPE
  msg.match = match
  msg.hard_timeout = 0
  msg.idle_timeout = 200
  msg.priority = priority + offset
  event.connection.send(msg)


 # Cuneyt Terzi: the Task 3.0 pseudocode (one MAC <-> one IP).
 # Returns True if the source MAC is blocked
 def portSecurity(self, match, event):
  print("Entering portSecurity")

  # Only inspect IP packets (they carry a source IP we can check).
  if match.dl_type != pkt.ethernet.IP_TYPE:
    return False
  if match.nw_src is None:
    return False

  src_mac = str(match.dl_src)
  src_ip  = str(match.nw_src)

  # if attacjer is already in the list, drop packet
  if src_mac in self.blockedMACs:
    return True

  # add the ip info to the list
  seen_ips = self.portTable.setdefault(src_mac, set())
  if src_ip not in seen_ips:
    seen_ips.add(src_ip)
    print("PORT SECURITY: MAC %s now maps to IPs %s",
          src_mac, list(seen_ips))

  # if more then one ip, block the mac address and also drop
  if len(seen_ips) > self.maxIPsPerMAC:

    print("Blocking MAC %s (used %d IPs)", src_mac, len(seen_ips))
    self.blockMAC(src_mac, event)
    return True

  return False


 # Cuneyt Terzi: block a spoofing attempt by adding a rule to l2firewall.config with top priority
 def blockMAC(self, src_mac, event):
  print("PORT SECURITY: BLOCKING spoofing MAC %s (used %d IPs)",
        src_mac, len(self.portTable.get(src_mac, set())))

  msg = of.ofp_flow_mod()
  m = of.ofp_match()
  m.dl_src = EthAddr(src_mac)
  msg.match = m
  msg.priority = 65535          # highest priority
  # No actions appended => the switch drops matching packets.
  event.connection.send(msg)
  print("Adding MAC %s to blockedMACs set", src_mac)
  self.blockedMACs.add(src_mac)

  # write to l2firewall.config -> id,mac_0,mac_1 (mac_1 'any' = any dest).
  print("Writing to l2firewall.config: 100,%s,any", src_mac)
  try:
    with open(l2config, 'a') as fh:
      fh.write("100,%s,any\n" % src_mac)
  except Exception as err:
    print("PORT SECURITY: could not write l2firewall.config: %s", err)

 def replyToIP(self, packet, match, event, fwconfig):
  srcmac = str(match.dl_src)
  dstmac = str(match.dl_src)
  sport = str(match.tp_src)
```

```
dport = str(match.tp_dst)
nwproto = str(match.nw_proto)

with open(l3config) as csvfile:
  print("Reading l3config file in replyToIP")
  self.rules = csv.DictReader(csvfile)
  for row in self.rules:
    prio = row['priority']
    srcmac = row['src_mac']
    dstmac = row['dst_mac']
    s_ip = row['src_ip']
    d_ip = row['dst_ip']
    s_port = row['src_port']
    d_port = row['dst_port']
    nw_proto = row['nw_proto']

    print("You are in original code block ...")
    srcmac1 = EthAddr(srcmac) if srcmac != 'any' else None
    dstmac1 = EthAddr(dstmac) if dstmac != 'any' else None
    s_ip1 = s_ip if s_ip != 'any' else None
    d_ip1 = d_ip if d_ip != 'any' else None
    s_port1 = int(s_port) if s_port != 'any' else None
    d_port1 = int(d_port) if d_port != 'any' else None
    prio1 = int(prio) if prio != None else priority
    if nw_proto == "tcp":
      nw_proto1 = pkt.ipv4.TCP_PROTOCOL
    elif nw_proto == "icmp":
      nw_proto1 = pkt.ipv4.ICMP_PROTOCOL
      s_port1 = None
      d_port1 = None
    elif nw_proto == "udp":
      nw_proto1 = pkt.ipv4.UDP_PROTOCOL
    else:
      print("PROTOCOL field is mandatory, Choose between ICMP, TCP, UDP")
    print (prio1, s_ip1, d_ip1, s_port1, d_port1, nw_proto1)
    self.installFlow(event, prio1, srcmac1, dstmac1, s_ip1, d_ip1, s_port1, d_port1, nw_proto1)
self.allowOther(event)

def  handle ConnectionUp (self, event):

self.connection = event.connection

for (source, destination) in self.disabled_MAC_pair:

  print (source, destination)
  message = of.ofp_flow_mod() # OpenFlow message. Instructs a switch to install a flow
  match = of.ofp_match() # Create a match
  match.dl_src = source # Source address

  match.dl_dst = destination # Destination address
  message.priority = 65535 # Set priority (between 0 and 65535)
  message.match = match
  event.connection.send(message) # Send instruction to the switch

print("Firewall rules installed on %s", dpidToStr(event.dpid))

def _handle_PacketIn(self, event):
print("Entering _handle_PacketIn")

packet = event.parsed
match = of.ofp_match.from_packet(packet)
print("PacketIn: %s" % (match,))
if(match.dl_type == packet.ARP_TYPE and match.nw_proto == arp.REQUEST):

  self.replyToARP(packet, match, event)

if(match.dl_type == packet.IP_TYPE):

  # Cuneyt Terzi: check with port security
  # packet blocked if mac matches more than one IPs
  if self.portSecurity(match, event):
    print("PORT SECURITY: BLOCKING spoofing MAC %s (used %d IPs)")
    return

  ip_packet = packet.payload
  print ("Ip_packet.protocol = ", ip_packet.protocol)
```

```
      if ip_packet.protocol == ip_packet.TCP_PROTOCOL:
        print("TCP")

      # self.replyToIP(packet, match, event, self.rules)


def launch (l2config="l2firewall.config", l3config="l3firewall.config"):
    '''
    Starting the Firewall module
    '''
    print("Starting the Firewall module with l2config: %s and l3config: %s" % (l2config, l3config))
    parser = argparse.ArgumentParser()
    parser.add_argument('--l2config', action='store', dest='l2config',
            help='Layer 2 config file', default='l2firewall.config')
    parser.add_argument('--l3config', action='store', dest='l3config',
            help='Layer 3 config file', default='l3firewall.config')
    core.registerNew(Firewall, l2config, l3config)
```